

Assignment 2: n -gram Language Models on BPE Tokens

1. Goal

Your group builds an n -gram language model. The model reads the BPE tokens from Assignment 1. It learns the probability of the next token from the previous tokens. You build the unigram model first, then the bigram model, then the trigram model. You measure each model with perplexity. You then use the model to generate text.

The engine must work for $n = 1$, $n = 2$ and $n = 3$. Do not go above $n = 3$.

2. Rules

1. Write the model yourselves. Use only the Python standard library and `numpy` for it. Do not use `nltk.lm`, `kenlm` or a similar library.
2. Use your own BPE tokenizer from Assignment 1. If your tokenizer had a bug, fix it first.
3. Use add-one (Laplace) smoothing. Apply it always, to every count. Do not implement any other smoothing method.
4. Use the validation set to choose the order n of the n -gram model and the vocabulary size k of the BPE tokenizer. Choose the pair with the lowest validation perplexity. Do not use the test set for this choice.
5. Make the run deterministic. Set the random seed for sampling and report it.
6. You may use an AI assistant. If you do, write one paragraph in the README that explains how it played a role in your implementation.

3. Data

Use the same Shakespeare corpus as in Assignment 1. Clean it with the `clean` normalization strategy from Assignment 1. Then split it into a training set, a validation set and a test set yourselves. Split by line. Do not shuffle. Choose the exact percentages and justify them in the notebook; a typical split uses about 80 % for training, 10 % for validation and 10 % for test. Keep the split deterministic, so a second run gives the same three files.

Each line of a file is one sequence. Treat a line as follows:

1. Encode the line with your BPE tokenizer.
2. Put $n - 1$ tokens `<bos>` in front of the sequence.
3. Put one token `<eos>` at the end of the sequence.

The model must learn to predict `<eos>`. Later, the generator stops when it produces `<eos>`. Add `<bos>`, `<eos>` and `<unk>` to the vocabulary. The vocabulary size V counts these three tokens.

4. Task 1: Prepare the data and tokens (15 points)

1. Split the corpus into the training, validation and test sets, as described above. Print the number of lines and the number of characters in each part.
2. Train a BPE tokenizer on the training set. Any reasonable value of k is fine for now; you search over k in Task 4.
3. Encode all three splits as sequences of token ids, with `<bos>` and `<eos>`.
4. Print the number of tokens in the validation split and the test split that map to `<unk>`. This number should be small.

5. Task 2: The n -gram engine (30 points)

Write a class `NGramLM`. It takes the order n and the vocabulary size V .

1. `fit(sequences)` counts every n -gram and every $(n - 1)$ -gram in the training sequences. Keep the counts in dictionaries.

2. `log_prob(token, context)` returns $\log P(\text{token} \mid \text{context})$. The context is the list of the previous $n - 1$ tokens. Use add-one smoothing:

$$P(w \mid h) = \frac{c(h, w) + 1}{c(h) + V}$$

Here $c(h, w)$ is the count of the n -gram and $c(h)$ is the count of its history. Apply this formula to every token, also to tokens that the model has seen. Do not add a special case for a zero count.

3. `next_token_log_probs(context)` returns a `numpy` array with one entry for each token id in the vocabulary. The exponentials of the entries must sum to 1.
4. A token in the context that never appears in the training counts gets no history count. Give it the average unigram probability, which is $1/V$. Say in the notebook how you handle this case.
5. For $n = 1$ the context is empty. The class must still work.

Check your engine on a toy corpus of five short sequences before you use the real data. Compute two probabilities by hand and compare them against the code.

6. Task 3: Perplexity (15 points)

Write a function `perplexity(model, sequences)`. It takes any model with a `log_prob` method. It returns:

$$\text{PP} = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid h_i)\right)$$

Here N counts every predicted token, including `<eos>`. It does not count `<bos>`, because the model does not predict `<bos>`. Sum the log probabilities. Do not multiply the probabilities.

Also return the perplexity per character:

$$\text{PP}_{\text{char}} = \exp\left(-\frac{1}{C} \sum_{i=1}^N \log P(w_i \mid h_i)\right)$$

Here C is the number of characters in the original text. The two values answer different questions. The perplexity per token depends on k , because a larger k gives longer tokens. The perplexity per character does not. Use the second value whenever you compare across k .

Sanity check. Compute the perplexity of the unigram model on its own training data. Then compute it on a file of 200 random token ids. Explain the two numbers in one sentence each.

7. Task 4: Model selection on the validation set (20 points)

Use the tool from Task 3 to choose the order n and the vocabulary size k .

1. For $k \in \{250, 500, 1000, 2000, 4000\}$, train a BPE tokenizer on the training split. For each k , encode the training split and the validation split.
2. For each k and for $n \in \{1, 2, 3\}$, train an `NGramLM` with add-one smoothing on the training tokens. This gives 15 models.
3. Compute the perplexity per character of every model on the validation set. Report the full grid as one table, with n in the rows and k in the columns.
4. Plot the perplexity per character against k , with one line for each n . Use a logarithmic x axis.
5. Choose the pair (n^*, k^*) with the lowest validation perplexity per character. State the choice and comment on the shape of the three curves in two sentences.

Do not use the test set in this step. Once you have chosen (n^*, k^*) , evaluate that one model on the test set. Report both perplexity variants. Also report the test perplexity of the other two values of n at k^* , so the benefit of a higher order is visible in the same table.

8. Task 5: Generation (20 points)

Write a function `generate(model, tokenizer, prompt, mode, max_tokens, seed)`.

1. Normalize the prompt with the same strategy as the training data. Encode it with the tokenizer. Put $n - 1$ tokens `<bos>` in front.
2. Take the last $n - 1$ tokens as the context. Call `next_token_log_probs`.
3. Choose the next token. In mode `argmax`, take the token with the highest probability. In mode `sample`, draw one token from the distribution.
4. Append the token. Stop when the token is `<eos>`. Also stop after `max_tokens` tokens, and say in the output that the limit stopped the generation.
5. Decode the token ids with the tokenizer. Return the text.

Use the final model from Task 4. Use the same three prompts for every mode. Generate ten outputs per mode. Show them in the notebook. Then answer these two points in a short table: how many `argmax` outputs fall into a loop, and how many `sample` outputs stop at `<eos>` before the limit.

The function `generate` must work with any model that has `next_token_log_probs`. Do not put *n*-gram code inside it.

9. Required interface

The notebook is the only thing that we run. Keep the code in `.py` files next to it, or in the notebook. Use these names:

```
from bpe import BPETokenizer
from ngram import NGramLM
from evaluation import perplexity, generate

tok = BPETokenizer.load("models/bpe_1000.json")
train_ids = encode_file(tok, "data/train.txt", n=3) # list of id sequences

lm = NGramLM(n=3, vocab_size=len(tok.vocab))
lm.fit(train_ids)
lm.log_prob(token_id, context_ids)
lm.next_token_log_probs(context_ids) # numpy array, length V

pp, pp_char = perplexity(lm, test_ids)
text = generate(lm, tok, "shall i compare thee", mode="sample", max_tokens=50, seed=0)
```

Keep `perplexity` and `generate` separate from the model. You use both again in Assignment 3.

10. What you hand in

The deadline for all submissions is 30 September 2026 at 17:00. This applies to Assignment 1, Assignment 2 and Assignment 3.

Make one ZIP file with the name `group_NN_a2.zip`. Upload it to the folder of your group on StudIP. One member uploads for the whole group. The ZIP contains:

- `assignment2.ipynb`, with all output cells kept. It must run from top to bottom after *Restart and Run All*.
- Your `.py` files, including `bpe.py` from Assignment 1, and `requirements.txt`.
- The saved BPE files under `models/`. Keep the ZIP under 10 MB.
- `README.md`. See the next paragraph.

The contribution statement. The `README.md` must start with a table in this form:

Name	Student number	Tasks	Share
Student 1	123456	Task 2, toy corpus test	40 %
Student 2	234567	Task 3, Task 4, plots	35 %
Student 3	345678	Task 1, Task 5, README	25 %

The shares must sum to 100. All members must agree to the table. Add one sentence about the work of each member in your own words. We grade the submission as one piece of work, but we use this table in the review session, and we can adjust a single grade if the table and the review do not agree. A submission without this table loses 10 points.

The `README.md` also gives the total run time, the machine that you used, the random seed, the percentages that you chose for the data split, and one paragraph on how AI played a role in your implementation, if you used it. Do not hand in the corpus. Do not hand in empty output cells.

11. Grades

Item	Points
Task 1. Data split, tokens with <code><bos></code> and <code><eos></code> , counts	15
Task 2. Correct n -gram engine with add-one smoothing, toy corpus check	30
Task 3. Perplexity function, both variants, sanity check	15
Task 4. Model selection on the validation set, final test evaluation	20
Task 5. Generator with both modes, stop at <code><eos></code> , examples	20
Total	100

We remove points for these problems: the notebook does not run from top to bottom, -15 ; the `.py` files do not import, -10 ; the contribution table is absent, -10 ; the test set is used to choose n or k , -10 ; `perplexity` or `generate` contains n -gram code, -5 ; two runs give different numbers, -5 ; a plot has no axis label or no legend, -2 each.

12. The next assignment

Assignment 3 is the last assignment in the course. It appears this Friday. It is a small GPT: you build a transformer language model in PyTorch. It reads the same BPE tokens as the n -gram model, with the same `<bos>` and `<eos>`. It gives the same `next_token_log_probs` interface. So your `perplexity` function and your `generate` function work on it without a change. That is the reason why this assignment keeps them separate from the model. You then put the GPT next to the n -gram model in one table and one set of generated texts. Keep your code. You will import it again.